Principles - Categories

Learnability
the ease with which new users can begin effective interaction and achieve maximal performance

Flexibility
the multiplicity of ways the user and system exchange information

Robustness
the level of support provided the user in determining successful achievement and assessment of goal-directed behaviour

Principles supporting Learnability

Table 7.1 Summary of principles affecting learnability

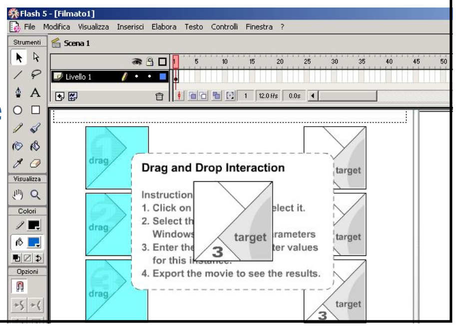
Principle	Definition	Related principles
Predictability	Support for the user to determine the effect of future action based on past interaction history	Operation visibility
Synthesizability	Support for the user to assess the effect of past operations on the current state	Immediate/eventual honesty
Familiarity	The extent to which a user's knowledge and experience in other real-world or computer-based domains can be applied when interacting with a new system	Guessability, affordance
Generalizability	Support for the user to extend knowledge of specific interaction within and across applications to other similar situations	–
Consistency	Likeness in input–output behavior arising from similar situations or similar task objectives	–

Predictability

If we know the interaction history are we able to determine the result of future interactions?

How much do we need to remember?
Every previous keystroke (difficult)
or nothing other than what is currently observable?

Example. Does the visual image on the screen indicate what objects form a compound object that can only be selected as a group?



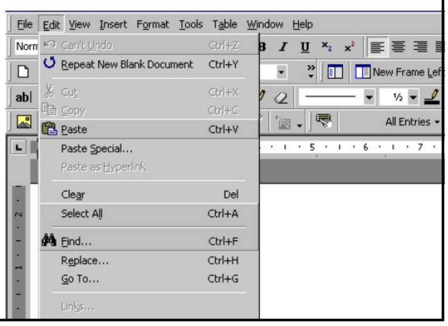
Operation Visibility

Principle	Definition	Related principles
Predictability	Support for the user to determine the effect of future action based on past interaction history	Operation visibility

Another form of predictability (related principle)

If an operation can be performed, is there some perceivable indication of this?

Recognizing is easier than recalling.
Affordance/logical constraints should be used to indicate available actions



Synthesizability (sinteticità)

Are we able to assess the effect of past operations on the current state?

When an operation changes some aspect of the internal state, it is important that
the change is seen by the user
→ principle of honesty

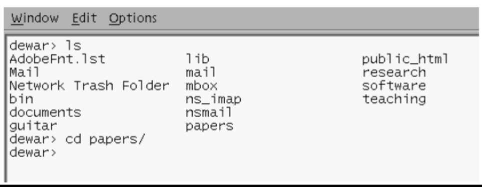
Honesty (trasparenza)

	future action based on past interaction history	
Synthesizability	Support for the user to assess the effect of past operations on the current state	Immediate/eventual honesty
Familiarity	The extent to which a user's knowledge and	Guessability,

When an operation changes some aspect of the internal state, is the user interface able to provide an <u>observable and informative account</u> of such change?

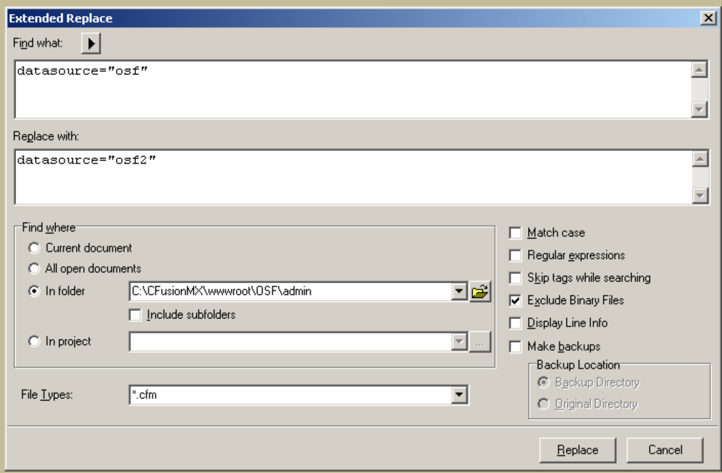
- **immediate honesty**: notification is immediate
- **eventual honesty**: notification requires other interactions

Example.
Command line interfaces are never honest.

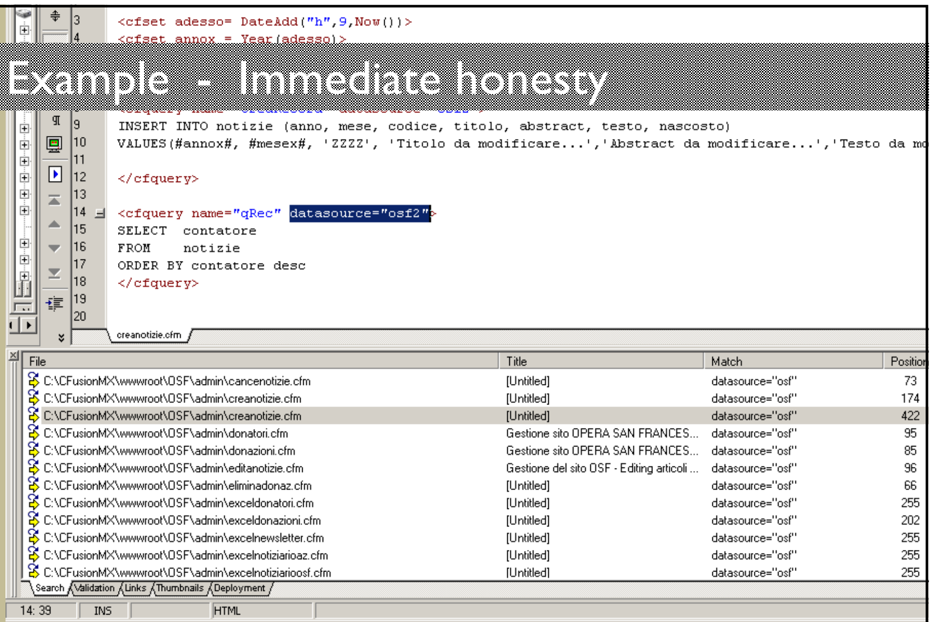

```
Window  Edit  Options
dewar> ls
AdobeFnt.lst          lib         public_html
Mail                  mail        research
Network Trash Folder  mbox        software
bin                   ns_imap     teaching
documents             nsmail
guitar                papers
dewar> cd papers/
dewar>
```

Example - Immediate honesty

Homesite editor - Extended Replace



Example - Immediate honesty



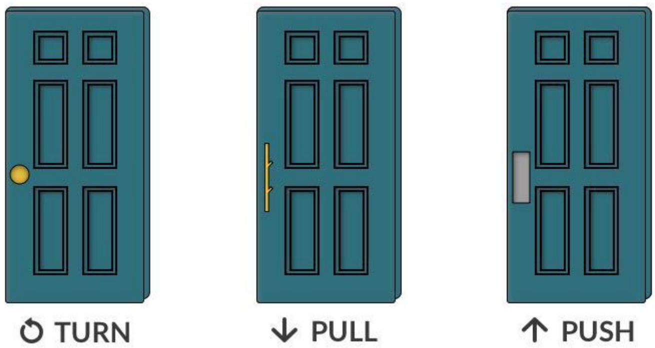
Familiarity

Are our experiences (obtained by interacting with the real world and computer systems) useful to understand how to interact with this system?

First impression?

Examples:
use of metaphor (e.g. tab-stops in word-processor)
use of natural language syntax, **affordances**

Affordance



Some psychologists argue that there are intrinsic properties, or affordances, of any visual object that suggest to us how they can be manipulated

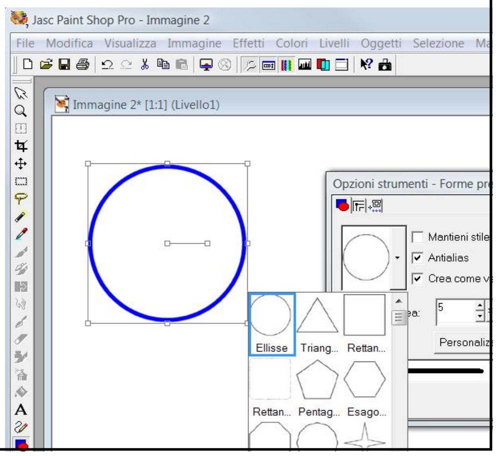
Generalizability

Are we able to extend specific interaction knowledge to new situations within this system?

It helps **give a predictive model** of system for user

It is a form of **consistency.** It can occur within a **single application** or across a **variety of applications**

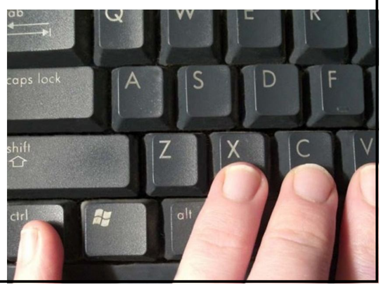
Examples: drawing circles as constrained form of ellipses.. and squares?



Generalizability and standards

UI standards and guidelines assist/enforce generalizability

Applications should offer the Cut/Copy/Paste operations whenever possible **in the same way**



Consistency (coerenza)

Probably the most widely mentioned principle:
"Be consistent!"

Consistency **is not self contained**:
- consistency within screens
- consistency within applications
- consistency within desktop

Many other principles can be 'reduced' to qualified instances of consistency.

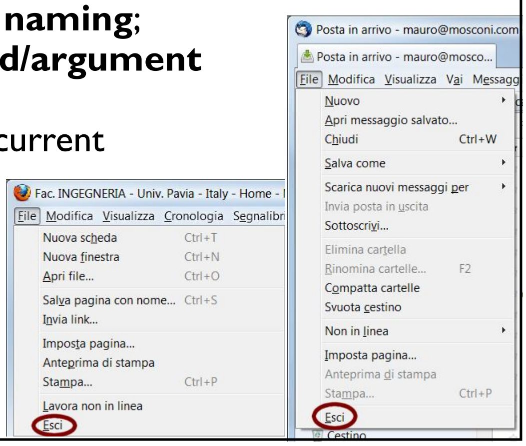
Are we able to recognize and use the same input/output forms seen in similar situations o tasks?

Consistency Examples

- consistent patterns in **layout**;
- **same short-cut keys** for similar action;
- consistency in **color codes**;
- consistency in command **naming**;
- consistency in **command/argument invocation;**
- same **placement** for recurrent menu options;

…

Always place the Quit command as the last item in the leftmost menu



Principles supporting Flexibility

Table 7.2 Summary of principles affecting flexibility

Principle	Definition	Related principles
Dialog initiative	Allowing the user freedom from artificial constraints on the input dialog imposed by the system	System/user pre-emptiveness
Multi-threading	Ability of the system to support user interaction pertaining to more than one task at a time	Concurrent vs. interleaving, multi-modality
Task migratability	The ability to pass control for the execution of a given task so that it becomes either internalized by the user or the system or shared between them	–
Substitutivity	Allowing equivalent values of input and output to be arbitrarily substituted for each other	Representation multiplicity, equal opportunity
Customizability	Modifiability of the user interface by the user or the system	Adaptivity, adaptability

Dialog Initiative (User Pre-Emptiveness)

Who controls dialogue flow? **Are users free to initiate any action towards the system?**

This type of dialog, called **user pre-emptive**, **favours** flexibility.

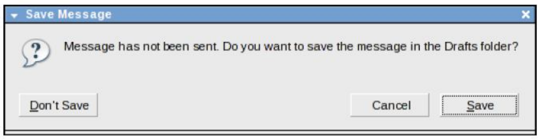
If only the system can initate the conversation (and the user simply responds), this dialog is called **system pre-emptive**.

Dialog Initiative Examples

User should be able to abandon, suspend or resume tasks at any point

Minimise system pre-emptive dialogue and maximise user pre-emptive dialogue!

Modal dialog boxes are system pre-emptive
Direct manipulation is user pre-emptive



When is system pre-emptiveness desirable?

Dialog Initiative Examples

A system pre-emptive dialog is required…
- in certain **cooperative** tasks
- for **safety** reasons

Moreover..
user pre-emptivness increases the likelihood that the **user will lose track** of the tasks that have been initiated and not yet completed

Multi-threading

A **thread** of a dialog is a coherent subset of that dialog. (e.g. that part of the dialog that relates to a given user task)

Multi-threading of the user–system dialog allows for interaction to support more than one task at a time.

This favours flexibility

Multi-threading

Concurrent multi-threading allows simultaneous communication of information pertaining to **separate tasks**.

Interleaved multi-threading permits a temporal overlap between separate tasks, but stipulates that at any given instant the dialog is restricted to a **single task**.

Multi-modality

Principle	Definition	Related principles
Multi-threading	Ability of the system to support user interaction pertaining to more than one task at a time	Concurrent vs. interleaving, multi-modality

Separate modalities (or channels of communication) are combined to form a single input or output expression

First case

Multiple channels may be available, but **any one expression** may be restricted to **just one channel** (keyboard or audio, for example).

Eg. to open a window:
double click on an icon
OR keyboard shortcut
OR vocal command

open window !

Multi-modality

Second case

A single expression can be formed by a mixing of channels

E.g. error warnings
(text + beep)

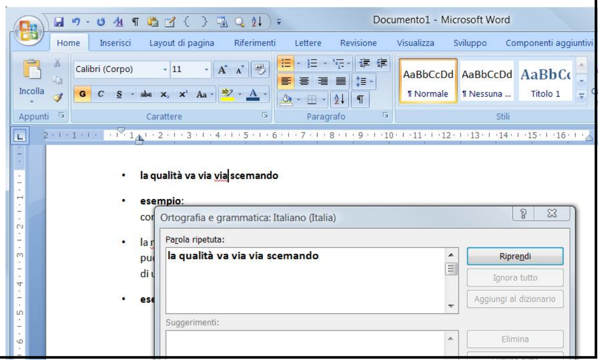
Task migratability (migrabilità dei compiti)

Can functions be easily moved between user and system?

People get bored doing routine tasks and stop concentrating (Yerkes-Dodson Law)

Automate routine tasks, but don't fix function allocation!

Example: spell-checking is best performed in a cooperative way.

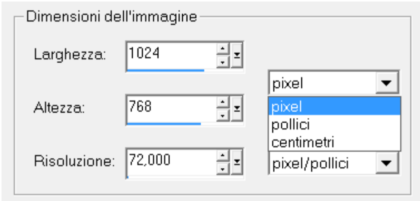


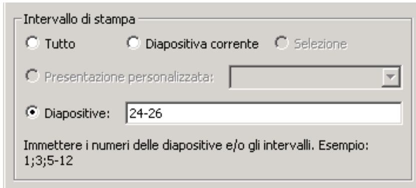
Task migratability - People vs Machines

People are better at	**Machines are better at**
Detecting small sensory inputs	Responding quickly to signals
Improvising and using flexible procedures	Following procedures repeatedly and precisely
Reasoning inductively	Reasoning deductively
Selective information recall	Total information recall
Exercising judgement	Following orders

Substitutivity

Are we allowed to substitute for each other equivalent values of input and output (in order to use the most comfortable ones)?

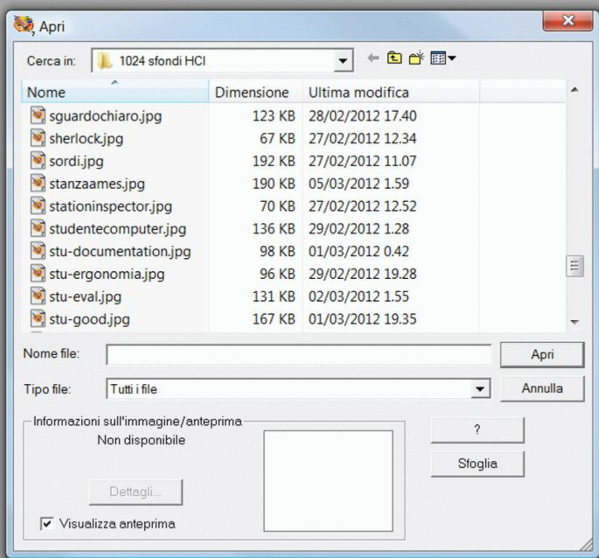




Substitutivity can **minimize user errors** and cognitive effort

Substitutivity

Don't force users to refer to objects by name if they can point to them!



Representation Multiplicity

Are output data represented in different ways, so that, at a given time, we (users) are free to consider the representations that are most suitable for the current task?

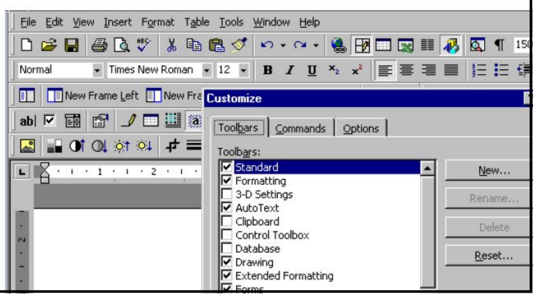
It might even be desirable to make these representations **simultaneously** available to the user.

Customizability

Customizability is the modifiability of the user interface by the user (adaptability) or by the system (adaptivity)

Provide choice of methods!
Allow shortcuts! Permit users to change features!

Adaptivity is automatic..
Decisions for adaptation can be based on user expertise or observed repetition of certain task sequences



Principles supporting Robustness

Table 7.3 Summary of principles affecting robustness

Principle	Definition	Related principles
Observability	Ability of the user to evaluate the internal state of the system from its perceivable representation	Browsability, static/dynamic defaults, reachability, persistence, operation visibility
Recoverability	Ability of the user to take corrective action once an error has been recognized	Reachability, forward/backward recovery, commensurate effort
Responsiveness	How the user perceives the rate of communication with the system	Stability
Task conformance	The degree to which the system services support all of the tasks the user wishes to perform and in the way that the user understands them	Task completeness, task adequacy

Observability (see Norman's model)

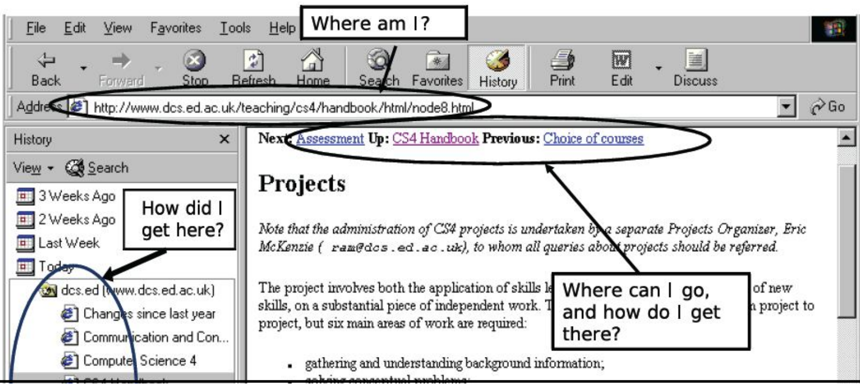
Is the user able to evaluate the internal state of the system by means of its perceivable representation at the interface?

Compare the current observed state with his intention…

Obtained through other principles:
- **browsability**,
- **defaults**,
- **persistence**
- **reachability**
- **operation visibility** (*)

Example - Where³ What of navigation

- **W**here am I? — immediate honesty wrt system state
- **W**here am I going? — operation predictability
- **W**here have I been? — synthesisability
- **W**hat can I do now? — predictability



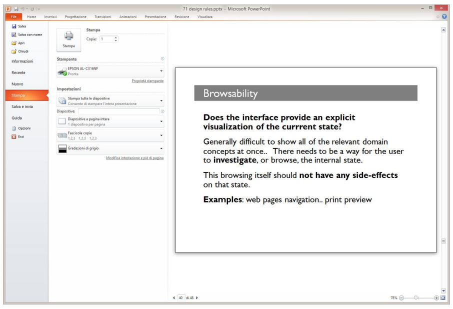
Browsability

Does the interface provide an explicit visualization of the current state?

Generally difficult to show all of the relevant domain concepts at once.. There needs to be a way for the user to **investigate**, or browse, the internal state.

This browsing itself should **not have any side-effects** on that state.

Examples: web pages navigation.. print preview

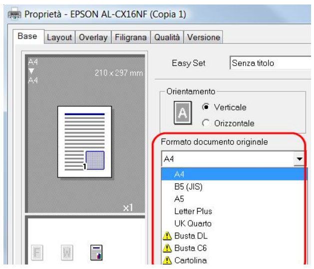


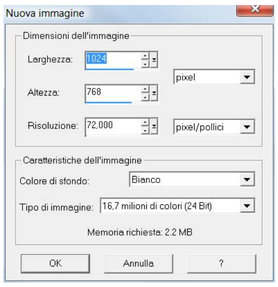
Defaults

Does the interface help users recognizing rather than recalling?

Defaults..
1) **reduce the number of physical actions** necessary to input a value
2) are a kind of **error prevention mechanism**

Two kinds of default values:
- **static** defaults do not evolve with the session
- **dynamic** defaults evolve during the session (computed from previous inputs)





Persistence

The system informs the user on its internal state. How long does the effect of this notice persist in the user's memory? Enough for it to be exploited?

The effect of **vocal communication** does not persist.. **Visual communication**, on the other hand, can remain as an object..

Recoverability

Is the user able to take corrective action once an error has been recognized?

Two directions:

Forward error recovery involves the acceptance of the current state and negotiation from that state towards the desired state.

Backward error recovery is an attempt to undo the effects of previous interaction in order to return to a prior state before proceeding.

Recovery can be initiated by the system or by the user.

Recoverability

- supported by reducing scope for making errors
  - *avoid free-form input where possible*
  - *validate input immediately, allowing correction*
- . . . and ability of user to understand errors
  - *error messages should be concise, informative, specific, constructive*

Poor:	Error 404: document not found
Better:	The requested URL `http://www.foobar.com/bar.html` could not be found

Commensurate effort

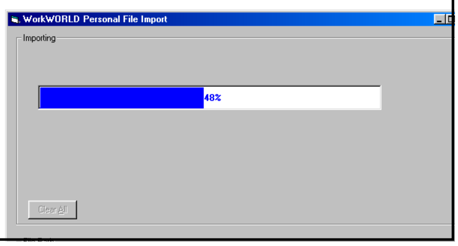
If it is difficult to undo a given effect on the state, then it should have been difficult to do in the first place.

Conversely,
easily doable actions should be easily undone.

Responsiveness - Response Time Stability

Response time is generally defined as the duration of time needed by the system to express state changes to the user. **Does the user perceives system reactions as immediate?**

If an instantaneous response cannot be obtained, there must be **some indication** to the user that the system has received the request for action and is working on a response.



Response time stability: invariance of the duration for identical or similar computational resources.

Task conformance

Does the system allow the user to achieve any of the desired tasks as identified by a task analysis?

- Few general purpose commands, long methods, simple
- Many highly tuned commands, short methods, complex
- *identify core tasks; provide a command for each*
- But core task set grows over time; language is cluttered as lexicon expands
  - e.g., Unix command language, once a small set now has >700; 10% account for 90% of usage.
  - Microsoft Word command lexicon now includes text formatting, drawing, annotating, WWW related commands, etc.

Use a principle to provide a usability specification

Example. Our product is an **electronic calendar**. We plan to increase the usability of a frequent task: scheduling weekly meetings…

Attribute	Task migratability
Measuring concept	Scheduling a weekly meeting
Measuring method	Time it takes to enter a weekly meeting appointment
Now level	(Time to schedule 1 appoint.) x (Number of weeks)
Worst case	Time to schedule 2 appointments
Planned level	1.5 x (Time to schedule 1 appointment)
Best case	Time to schedule 1 appointment